

Guia de estudo do código

Este guia foi pensado para quem está começando em programação e quer estudar o projeto com menos ansiedade. A ideia não é entender tudo de uma vez, mas seguir uma ordem de leitura que faça sentido.

Regra prática: primeiro entenda **o que o programa faz**, depois **como ele calcula**, e só depois **como a interface conversa com esses cálculos**.

1. Por onde começar

Passo 1

Leia primeiro a versão PC

A versão local em Ondas_e_Cores.py é mais fácil de entender porque quase tudo está concentrado dentro de uma classe só.

Passo 2

Depois vá para a versão web

app_web.py reaproveita a mesma ideia física, mas espalha a lógica em várias funções e callbacks.

Passo 3

Compare os dois arquivos

Depois de entender o básico, compare os nomes das funções equivalentes nas duas versões.

2. Ordem recomendada para Ondas_e_Cores.py

1. Comece por `__init__` para ver quais dados o programa guarda.
2. Leia `_setup_plots`, `_setup_sliders` e `_setup_controls` para entender a montagem da interface.
3. Leia `_generate_wave` para descobrir como a onda é construída.
4. Leia `_compute_envelope` e `_max_envelope_difference` para entender o critério principal de comparação.
5. Leia `_update_plots` para ver como tudo se junta na tela.
6. Leia `_run_optimization` por último entre as partes numéricas, porque ele usa quase tudo que veio antes.
7. Depois visite as janelas auxiliares: Fourier, Hilbert e Lab RGB.

Objetivo dessa ordem: primeiro enxergar a estrutura, depois a matemática, e só então a parte mais algorítmica.

3. Ordem recomendada para app_web.py

1. Leia as constantes do topo para reconhecer frequências, tempos, limites e nomes importantes.
2. Leia as funções do núcleo numérico: `clamp_amp`, `generate_wave`, `compute_envelope` e `optimize_amplitudes`.
3. Leia as funções visuais: `build_main_figure`, `build_intensity_figure`, `build_summary` e `build_rgb_card`.
4. Leia as funções de sessão: `create_exercise_session`, `normalize_exercise_session` e `evaluate_exercise`.
5. Leia as funções de planilha: `build_final_session_payload`, `write_class_sheet_worksheet` e `send_final_session_results`.
6. Deixe os callbacks do Dash para o final: `handle_actions`, `refresh_outputs`, `handle_exercises` e `handle_final_results_send`.

Os callbacks ficam por último porque eles fazem mais sentido quando você já sabe o que cada função auxiliar faz.

4. O que observar em cada etapa

Quando ler a síntese de Fourier

Pergunte: como as amplitudes das frequências viram uma onda no tempo?

Quando ler Hilbert

Pergunte: por que o programa compara envelopes e não simplesmente as ondas brutas?

Quando ler a otimização

Pergunte: qual número o algoritmo está tentando diminuir e quais variáveis ele pode mexer?

Quando ler a parte web

Pergunte: qual função calcula algo e qual função apenas mostra esse resultado na interface?

5. Estratégia prática para estudar sem travar

- Não tente entender o arquivo inteiro em uma única leitura.
- Leia um método por vez e anote, em uma frase, o que ele recebe e o que ele devolve.
- Quando aparecer um nome desconhecido, procure onde ele foi definido antes de continuar.
- Separe mentalmente três camadas: matemática, interface e armazenamento de dados.
- Se algo parecer complicado demais, volte para a versão PC, que é a base conceitual mais simples.

6. Meta realista de aprendizagem

Uma meta muito boa para as próximas leituras é esta: conseguir explicar, com suas palavras, como uma lista de amplitudes vira duas ondas, como essas ondas viram envelopes e como a otimização tenta aproximar esses envelopes. Se você conseguir isso, já terá entendido o coração do projeto.